

Refactoring_document:**Shiah****Long Method in complete workout session****Identification**

Smell ID	Category	Severity	File
SMELL-001	Long Method / SRP Violation	High	backend/api/views.py

The complete_workout_session view function handled the following distinct responsibilities all in a single block of inline code:

- Parsing and validating the request date
- Creating or updating the WorkoutSession record
- Inferring session duration from program session_length
- Awarding points via _award_points
- Checking and awarding badges via _check_and_award_badges
- Updating challenge progress for every active UserChallenge

This violated the Single Responsibility Principle (SRP): a single function was doing the work of at least three collaborators. It also created a Shotgun Surgery risk, any change to challenge logic, points logic, or badge logic required editing this one massive function.

Before (Problem)

Inside complete_workout_session, the challenge progress update was written as ~50 lines of inline code:

```
# --- US 4.4: UPDATE CHALLENGE PROGRESS ---
from django.utils import timezone
from .models import UserChallenge
active_user_challenges = UserChallenge.objects.filter(
    user=user, challenge__is_active=True, is_completed=False
).select_related('challenge')
for uc in active_user_challenges:
    # ... ~50 lines criteria checking, badge awarding, point awarding
```

This made complete_workout_session over 120 lines long with multiple levels of nested logic, making it difficult to read, test, or modify independently.

Refactoring Applied

Extract Method refactoring was applied. The ~50 lines of challenge progress update logic were extracted into a dedicated helper function:

```
def _update_challenge_progress(user, session, newly_unlocked_badges, total_points):
```

"""

Updates progress for every active, non-expired, non-completed challenge the user has joined. Awards points and badges when all goals are met. Returns the (possibly updated) total_points value.

"""

This mirrors the existing helper pattern already established in the same file (_award_points and _check_and_award_badges). The main function now calls:

total_points = _update_challenge_progress(user, session, newly_unlocked_badges, total_points)

After (Result)

- complete_workout_session is reduced to its core orchestration responsibility.
- _update_challenge_progress is independently testable.
- Future changes to challenge logic touch only one function.
- The helper also fixes BUG-002 by adding the end_date_gte=today filter.

GitHub Issue: Code Smell #1: Extract _update_challenge_progress helper from complete_workout_session

Commit: Committed to main branch - Please see GitHub commit history

Magic Strings for Session Status Values

Identification

Smell ID	Category	Severity	File
SMELL-003	Magic Strings / Primitive Obsession	Medium	backend/api/views.py

Throughout views.py, session status values were referenced as raw string literals in at least 8 distinct locations across 6 different functions:

- 'completed' : in complete_workout_session, _analyze_feedback, get_progress_summary, workout_history
- 'in_progress' : in start_workout_session, complete_workout_session, undo_workout_session
- 'missed' : in auto_mark_missed_sessions, workout_history

Using raw string literals (magic strings) means a single typo; such as 'complete' instead of 'completed' , would cause a silent runtime bug that only testing would catch, not the interpreter. It also makes search-and-replace refactoring fragile and inconsistent.

Before (Problem)

Examples of magic strings scattered across the file:

```
status='missed',                # auto_mark_missed_sessions
defaults={"status": "in_progress", ...} # start_workout_session
if session.status != "completed":      # start_workout_session
    session.status = "completed"        # complete_workout_session
status__in=['completed', 'missed']     # workout_history
session_status='completed',            # analyze_feedback
status='completed'                    # get_progress_summary
```

Refactoring Applied

Three named constants were defined at the top of views.py, immediately after the existing DAYS_OF_WEEK constant:

```
SESSION_STATUS_IN_PROGRESS = 'in_progress'
SESSION_STATUS_COMPLETED = 'completed'
SESSION_STATUS_MISSED = 'missed'
```

Every magic string occurrence in the file was then replaced with the corresponding constant.

Examples of the replacements:

```
# auto_mark_missed_sessions, BEFORE: status='missed'
# AFTER:
status=SESSION_STATUS_MISSED,

# start_workout_session, BEFORE: defaults={"status": "in_progress"}
# AFTER:
defaults={"status": SESSION_STATUS_IN_PROGRESS, "is_completed": False},

# workout_history, BEFORE: status__in=['completed', 'missed']
# AFTER:
status__in=[SESSION_STATUS_COMPLETED, SESSION_STATUS_MISSED]
```

After (Result)

- Status values are defined once and referenced everywhere — no duplication.
- A typo in a status value is now detectable at import time via a NameError, not a silent query mismatch.
- Future changes to status values require editing exactly one line.
- All 8 replacement sites have been updated consistently.

GitHub Issue: Code Smell #3: Replace magic status strings with named constants

Commit: Committed to main branch.

Paid / Free Access Toggle

Story	US 1.3: Create Programs from the list of workouts
File	frontend/src/app/create-program/page.tsx
State	isPaid: boolean (default: false)
API field	is_paid sent in POST /api/programs/ payload

Before

All programs created by trainers were free by default. There was no UI control to mark a program as paid or subscription-only. The `is_paid` field existed on the backend model (`WorkoutPlan.is_paid`) but the create-program form never sent it, so it always defaulted to False.

After

A full-width animated toggle was added under an 'Access Type' label. Clicking it switches between Free and Paid / Subscription. The toggle uses inline styles for the animated sliding pill and colour-coded border. The `is_paid` value is included in the `programData` payload submitted to the backend on form save.

// State

```
const [isPaid, setIsPaid] = useState(false);
```

// Submitted in handleSubmit

```
const programData = {
  name: programName,
  ...
  is_paid: isPaid, // <-- new field
  sections: [...],
};
```

// Toggle UI (simplified)

```
<div onClick={() => setIsPaid(p => !p)}
  style={{ border: `2px solid ${isPaid ? '#f59e0b' : '#22c55e'}` }}>
  <span>{isPaid ? '🔒 Paid / Subscription' : '🔓 Free'}</span>
```

```

    { /* animated sliding pill */ }
  </div>

```

Success Toast with Navigation Shortcut

Story	US 1.3: Create Programs from the list of workouts
File	frontend/src/app/create-program/page.tsx
State	successMessage: string null
Replaces	Silent redirect or plain alert() on successful save

Before

On successful program creation, the page either redirected immediately or showed a browser alert(). Neither approach confirmed which program was saved, gave the trainer a way to stay on the page, or provided a link to view the new program.

After

A fixed-position green toast notification appears at the top-centre of the screen after a successful save. It displays a checkmark, the exact program name in quotes, a 'View Programs →' button that navigates to /trainer-programs, and a dismiss (X) button. The page also smoothly scrolls to the top so the toast is visible. The toast persists until dismissed, letting the trainer continue editing or navigate away.

// State

```
const [successMessage, setSuccessMessage] = useState<string | null>(null);
```

// In handleSubmit, on response.ok:

```
setSuccessMessage(`${programName} has been created successfully!`);
window.scrollTo({ top: 0, behavior: 'smooth' });
```

// Toast JSX

```
{successMessage && (
  <div style={{ position: 'fixed', top: '80px', left: '50%',
    transform: 'translateX(-50%)', background: '#22c55e', ... }}>
    <span>✔</span>
  </div>
)}
```

Group2

```
<span>{successMessage}</span>
  <button onClick={() => router.push('/trainer-programs')}>
    View Programs →
  </button>
  <button onClick={() => setSuccessMessage(null)}> ✕ </button>
</div>
}
```

Rest Day Confirmation Modal

Story	US 1.3: Create Programs from the list of workouts
File	frontend/src/app/create-program/page.tsx
State	showRestDayConfirm: boolean, pendingRestDayIndex: number null
Prevents	Accidental loss of exercises when toggling a day to rest

Before

When a trainer clicked the rest day toggle on a day that already had exercises, all exercises were silently deleted with no warning. This was a destructive action with no way to recover.

After

Clicking the rest day toggle now checks whether the day has exercises. If it does, a confirmation modal appears showing how many exercises will be lost and warning 'Marking as rest day will remove all exercises.' The trainer must explicitly click 'Mark as Rest Day' to confirm, or 'Cancel' to keep the exercises. Only if the day has no exercises does the toggle fire immediately without a modal.

// State

```
const [showRestDayConfirm, setShowRestDayConfirm] = useState(false);
const [pendingRestDayIndex, setPendingRestDayIndex] = useState<number | null>(null);

const toggleRestDay = (index: number) => {
  const day = daySections[index];
  // Guard: has exercises and trying to mark as rest
```

Group2

```
if (!day.is_rest_day && day.exercises.length > 0) {  
  setPendingRestDayIndex(index);  
  setShowRestDayConfirm(true); // show modal instead  
  return;  
}  
  
// Safe to toggle immediately  
const updated = [...daySections];  
updated[index].is_rest_day = !updated[index].is_rest_day;  
setDaySections(updated);  
};  
  
const confirmRestDay = () => {  
  // User confirmed — clear exercises and mark rest  
  updated[pendingRestDayIndex].is_rest_day = true;  
  updated[pendingRestDayIndex].exercises = [];  
  setDaySections(updated);  
  setShowRestDayConfirm(false);  
};
```

end

Code Smell CS-01: Hard-coded Value

Smell Type: Hard-coded Value (Magic String)

User Story: US 2.4

File: frontend/src/app/program/[id]/page.tsx (line 206)

GitHub Issue: #65

Commit: 8b7f730 on branch arshia/lab5-code-smells

What was checked:

Whether any API URLs were hard-coded instead of using environment variables.

What was detected:

The feedback fetch function had a hard-coded `http://localhost:8000` URL instead of using the `NEXT_PUBLIC_API_URL` environment variable. A code comment even explicitly said "BYPASSES BROKEN api.ts", confirming it was a known workaround left in the code.

Group2

Code Before (smell):

```
// DIRECT FETCH - BYPASSES BROKEN api.ts
const response = await
fetch(`http://localhost:8000/api/trainer/programs/${program.id}/feedback`, {
```

Code After (fix):

```
const response = await
fetch(`${process.env.NEXT_PUBLIC_API_URL}/api/trainer/programs/${program.id}/f
eedback`, {
```

Why this is a smell:

Hard-coding a URL ties the code to a single environment. It will break on any machine where the backend runs on a different host or port, including staging, production, or a teammate's machine.

Refactoring Applied:

Replaced `http://localhost:8000` with `${process.env.NEXT_PUBLIC_API_URL}` and removed the misleading comment.

Result After Refactoring:

The application now correctly retrieves the API base URL from environment configuration, improving portability, maintainability, and adherence to best practices.

Status: Fixed

Code Smell CS-02: Duplicate Code and Magic Number

Smell Type: Duplicate Code and Magic Number

User Story: US 4.5

File: frontend/src/app/create-challenge/page.tsx (lines 67–77)

GitHub Issue: #66

Commit: 8b7f730 on branch arshia/lab5-code-smells

What was checked:

The code was reviewed for duplicated logic and unexplained numeric literals (magic numbers), particularly in utility functions related to date handling.

What was detected:

Two functions `todayIso()` and `minEndDateIso()` both manually formatted a date to ISO string using the same verbose template literal. This identical formatting logic was duplicated across both functions. Additionally, the number 1 (representing "minimum 1 day in the future") appeared as an unexplained magic number with no context.

Code Before (smell):

Group2

```
const todayIso = () => {
  const d = new Date();
  return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2,'0')}-${
    String(d.getDate()).padStart(2,'0')}`;
};

const minEndDatelso = () => {
  const d = new Date();
  d.setDate(d.getDate() + 1);
  return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2,'0')}-${
    String(d.getDate()).padStart(2,'0')}`;
};
```

Code After (fix):

```
const MIN_CHALLENGE_DURATION_DAYS = 1;
const tolsoDateString = (d: Date): string => d.toISOString().split('T')[0];
const todayIso = () => tolsoDateString(new Date());
const minEndDatelso = () => {
  const d = new Date();
  d.setDate(d.getDate() + MIN_CHALLENGE_DURATION_DAYS);
  return tolsoDateString(d);
};
```

Why this is a smell:

Duplicated formatting logic means any future change must be made in two places.
The magic number 1 has no context a reader cannot tell whether it means
"1 day", "1 week", or something else without reading the surrounding code.

Refactoring Applied:

Extracted the date formatting into a shared helper `tolsoDateString()` and replaced
the magic number with a named constant `MIN_CHALLENGE_DURATION_DAYS`.

Result After Refactoring:

Date formatting logic is now defined once and reused. The constant name makes
the business rule is self-documenting.

Status: Fixed

Code Smell CS-03: Inconsistent Error Handling (alert() instead of UI Notification)

Smell Type: Inconsistent Error Handling / Poor UX Pattern

User Story: US 4.5

File: frontend/src/app/challenges/page.tsx (line 46)

GitHub Issue: #67

Commit: 8b7f730 on branch arshia/lab5-code-smells

What was checked:

Whether error handling was consistent across all pages — specifically whether
all pages used the same in-page Notification component for errors.

What was detected:

Every other page in the application (trainer-programs, create-challenge, dashboard, program detail) uses an in-page Notification component to show errors. However, the Challenges page used a raw browser alert() call for the join-challenge error. This is inconsistent, jarring, blocks all page interaction, and cannot be styled or dismissed gracefully.

Code Before (smell):

```
const handleJoin = async (id: number) => {
  setJoining(id);
  try {
    await challengeAPI.joinChallenge(id);
    router.push('/dashboard');
  } catch {
    alert('Failed to join'); // raw browser alert
  } finally {
    setJoining(null);
  }
};
```

Code After (fix):

```
const [errorMessage, setErrorMessage] = useState<string | null>(null);

const handleJoin = async (id: number) => {
  setJoining(id);
  try {
    await challengeAPI.joinChallenge(id);
    router.push('/dashboard');
  } catch {
    setErrorMessage('Failed to join challenge. Please try again.');
```

```
  } finally {
    setJoining(null);
  }
};
```

JSX Update:

```
{errorMessage && (
  <div className="notification error" role="alert">
    {errorMessage}
    <button onClick={() => setErrorMessage(null)}>×</button>
  </div>
)}
```

Why this is a smell:

alert() is a browser-level popup that blocks all page interaction, cannot be styled, and breaks consistency with every other page in the app. It is a well-known anti-pattern in modern web development.

Refactoring Applied:

Replaced alert() with a stateful errorMessage displayed as an in-page notification div, consistent with the rest of the application.

Result After Refactoring:

Error handling is now consistent across all pages. Users see a styled, dismissible in-page message instead of a blocking browser popup.

Status: Fixed

WeiQinSitu

Code Smell 001: Arrow Anti-Pattern (Bloater) in Session Completion

Identification

- Smell ID: SMELL-011
- Category: Bloaters / Long Method (Arrow Anti-Pattern)
- Severity: Medium
- File: backend/api/views.py

Inside the complete_workout_session function, the fallback logic used to determine a workout's default duration (if the user did not explicitly provide one) was written using deeply nested control flow (try -> if -> if -> if). This "Arrow Anti-Pattern" heavily bloated the API controller right before the critical gamification (points) logic was executed. Leaving deeply nested database lookups inside a main controller makes the code difficult to read, maintain, and test.

Before (Problem)

The single view function contained 14 lines of deeply nested schedule-parsing logic:

else:

try:

```
schedule = UserSchedule.objects.get(user=request.user, is_active=True)
day_name = target_date.strftime('%A').lower()
section_ids = schedule.weekly_schedule.get(day_name, [])
if not isinstance(section_ids, list):
    section_ids = [section_ids] if section_ids != 'rest' else []
```

```
if section_ids:
    section = ProgramSection.objects.select_related("program").get(id=section_ids[0])
    session.plan = section.program
    if session.duration_minutes in (None, 0):
        session.duration_minutes = section.program.session_length
except (UserSchedule.DoesNotExist, ProgramSection.DoesNotExist):
    pass
```

Refactoring Applied

The "Extract Method" refactoring technique was applied. A new, separate helper function named `_apply_fallback_schedule_data(session, user, target_date)` was created outside the main view. All of the deeply nested schedule lookup logic was moved into this helper. The 14 lines of bloated code inside `complete_workout_session` were then deleted and replaced with a single, clean function call.

After (Result)

- The main controller is now lean and strictly handles the request/response cycle and points awarding.
- The Arrow Anti-Pattern (deep indentation) has been completely flattened in the main view.
- The fallback schedule logic is now isolated, making it reusable and easier to unit test independently.

GitHub Issue: SMELL-011: Arrow Anti-Pattern - deeply nested control flow used instead of helper function in `complete_workout_session` (US 4.1)

Commit: Merged into the main branch via PR #45 (Refactor: Extracted nested schedule logic into `_apply_fallback_schedule_data...`)

Code Smell 002: Fragile Tests (Test Smells) in Workout History

Identification

- Smell ID: SMELL-010
- Category: Test Smells / Fragile Tests
- Severity: Medium
- File: `backend/tests/unit/test_schedules.py`

Inside the `WorkoutHistoryTests` class, the automated tests blindly counted the total number of sessions in the test database to verify workout history logic. With the addition of the background auto-marking logic for missed sessions (US 3.5), the total row count changes dynamically. This tight coupling to the global database state caused valid tests to falsely crash when unexpected "missed" sessions were generated in the background.

Before (Problem)

The legacy tests relied on hardcoded assertions that checked the absolute length of the response array:

```
# The test blindly checked the total length of the sessions array
self.assertEqual(len(response.data["sessions"]), 2)
```

Refactoring Applied

The test assertions were decoupled from the total database row count. The code was updated to explicitly filter the JSON response for completed sessions using a Python list comprehension. This ensures the test only evaluates the intended records, regardless of other background database changes.

After (Result)

- Tests are completely isolated from the side effects of newly generated missed sessions.
- The test suite is robust, preventing brittle, false-positive test failures.
- The test logic clearly communicates its intent by explicitly filtering: `completed_count = len([s for s in response.data["sessions"] if s["status"] == "completed"])`.

GitHub Issue: SMELL-010: Fragile Tests - hardcoded DB row counts used instead of status filtering in WorkoutHistoryTests (US 3.5)

Commit: Merged into the main branch (Fix: Refactored WorkoutHistoryTests to correctly filter auto-generated missed sessions)

PR-01 Code smell: duplicate post-workout feedback logic in dashboard and schedule pages.

Steps taken to solve the bug:

I opened `frontend/src/app/dashboard/page.tsx` and `frontend/src/app/schedule/page.tsx`.

I reviewed both files and marked the feedback-related code that was repeated in both places.

I identified the duplicated parts, including feedback state values, submit logic, edit and delete logic, and the feedback form UI.

I created a shared hook for the common feedback state and actions.

I moved the repeated feedback state and handler logic into that shared hook.

I created a shared feedback form component for the repeated input fields and buttons.

I created a shared feedback summary component for the saved feedback display.

I removed the duplicated feedback blocks from `dashboard/page.tsx`.

I replaced the removed dashboard code with imports of the shared hook and shared components.

I removed the duplicated feedback blocks from `schedule/page.tsx`.

I replaced the removed schedule code with imports of the shared hook and shared components.

I kept only the page-specific behavior inside each page, such as which workout was selected and when the form opened.

I tested the feedback flow from the dashboard page.

I tested the feedback flow from the schedule page.

I confirmed that submitting, editing, deleting, and displaying feedback still worked correctly after the refactor.

PR-02 Code smell: signup page uses a hardcoded backend URL instead of shared API configuration

Steps taken to solve the bug:

I opened frontend/src/app/signup/page.tsx.
I found the signup request that used a full hardcoded backend URL.
I removed the direct localhost backend URL from the signup page.
I opened frontend/src/library/api.ts.
I used the shared API configuration there instead of keeping the signup request tied to a page-level URL.
I created or updated the shared signup helper in api.ts.
I made sure the helper still sent the same signup request to the same endpoint.
I returned to signup/page.tsx.
I replaced the raw fetch call with the shared signup helper.
I kept the request payload and feature behavior the same so the user-facing flow would not change.
I tested regular user signup after the change.
I tested trainer signup after the change.
I tested invalid signup input to make sure validation errors still worked.
I checked that successful signup still redirected the user to the login page.

PR-03 Code smell: dashboard page defines its own hardcoded API base URL instead of using the shared API configuration

Steps taken to solve the bug:

I opened frontend/src/app/dashboard/page.tsx.
I searched for the local API base constant used by the page.
I found the hardcoded localhost API base URL inside the dashboard file.
I removed that local hardcoded constant from the page.
I opened frontend/src/library/api.ts.
I used the shared API configuration there instead of keeping the backend base URL inside the dashboard page.
I added or reused shared dashboard API helper functions in api.ts.
I returned to dashboard/page.tsx.
I replaced the old direct endpoint construction with calls to the shared dashboard helper functions.
I kept the actual endpoints the same so the dashboard behavior would still work the same way for the user.
I tested the dashboard after login.
I checked that the summary data still loaded correctly.
I checked the browser console and network tab to make sure there were no broken requests after the refactor.

Group2